Where Should the Work Go? Hardware/Software Partitioning and DMA Design for TinyML Inference on a RISC-V SystemC Platform

Abdelrahman Youssef · COE 407/661 System on Chip, KFUPM · Term paper proposal

1. Problem

A TinyML system-on-chip runs a small CPU next to a DNN accelerator. Adding the accelerator raises coupled design questions: which part of each layer runs in hardware and which stays in software, what kind of compute fabric to build (a rigid systolic mesh or a flexible, reconfigurable one), and how the operands reach the accelerator (CPU copies or DMA, and how the DMA shares the bus with the CPU). The compute fabric's cycle count alone does not answer them: layer lowering (im2col), transfers, the driver and the layers the accelerator cannot run stay on the CPU and the bus. This project will evaluate these questions together, at system level, in SystemC.

2. Anchor Paper

F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, T. Krishna, "STONNE: Enabling Cycle-Level Microarchitectural Simulation for DNN Inference Accelerators," IEEE Computer Architecture Letters, vol. 20, no. 2, pp. 122–125, 2021 [1].
What it does: builds rigid (TPU-like) and flexible (MAERI, SIGMA) accelerators from three configurable on-chip networks (distribution, multiplier, reduction), simulates them cycle by cycle, and is open source. *What this project will build on it:* STONNE will become the timing engine of a memory-mapped accelerator inside a SystemC/TLM-2.0 SoC with a RISC-V instruction-set simulator [3,5], so the accelerator will be evaluated together with the CPU, the bus, a DMA engine and the bare-metal driver that uses it.

3. Literature Review (scope)

About 15 papers in four groups: (a) accelerator simulation and design-space exploration: STONNE [1,2], ONNXim [11], ZigZag [13], SECDA [10]; (b) rigid versus flexible compute fabrics: TPU [9], MAERI [7], SIGMA [8]; (c) HW/SW co-design for microcontroller-class inference, including tiled, double-buffered data movement: DORY [12], XpulpNN [14], ESP4ML [15]; (d) SystemC virtual prototypes for RISC-V [3,4,5]. For each group, the review will note which system-level costs (transfers, lowering, unsupported layers, bus sharing) are evaluated.

4. Objective and Research Questions

Measure, on one SystemC platform and the MLPerf Tiny benchmark [6], how the end-to-end inference time changes from software only, to a simple accelerator, to a better accelerator, and then with different DMA and bus designs, and explain each change with a per-phase time breakdown.

- **RQ1 – Partition.** Once GEMMs are offloaded, how will the remaining time split between im2col, depthwise layers left on the CPU, data transfer and compute?
- **RQ2 – Fabric.** Does a flexible fabric improve end-to-end time with the same partition, and what distribution bandwidth does it need against a rigid 16×16 mesh?
- **RQ3 – Data movement.** How much do CPU copying versus DMA, and the DMA width, change the result? (Optional: when do pruned weights make SIGMA worthwhile?)
- **RQ4 – DMA and bus design.** Once the computation is offloaded, how much do overlapping transfers with compute (double buffering), a completion interrupt instead of polling, and the DMA's bus-ownership mode (burst, cycle stealing, transparent) change the end-to-end time and the CPU time left for other work?

5. Setup

- Course Docker image (Ubuntu 24.04, SystemC 2.3.3, C++20); simulation only, no hardware board needed.
- Platform: RISC-V VP++ (RV32IM ISS, SystemC/TLM-2.0) [3,5], to be extended with the accelerator: an SC_MODULE with a register file and scratchpads (TLM target) and a DMA engine (TLM initiator), with compute cycles from STONNE [1]; for RQ4 also an interrupt controller and an arbitrated shared bus.
- Workload: the three MLPerf Tiny INT8 models [6] (ResNet-8, DS-CNN, MobileNetV1-0.25): 48 layers, of which 31 are standard, pointwise or fully connected and 17 are depthwise.

6. Method and Steps

- **S0** Reproduce STONNE's rigid/flexible/sparse comparison on MLPerf Tiny layer shapes (STONNE alone).
- **S1** Software baseline: every layer in C on the ISS; cycles per layer.
- **S2** Simple accelerator: 16×16 TPU-like mesh; GEMMs offloaded, im2col and depthwise on the CPU; CPU copy and DMA.
- **S3** Better accelerator: (a) MAERI fabric with the same partition; (b) MAERI with native convolution, so im2col and depthwise move to hardware.
- **S4** Design-space exploration: mesh size, distribution bandwidth, DMA width, weight sparsity.
- **S5** DMA and bus: tiled, double-buffered DMA; interrupt-driven completion (PLIC, WFI); and a shared, arbitrated bus with burst, cycle-stealing and transparent DMA, plus a burst-size sweep.

Verification. The SystemC accelerator will be unit-tested on SystemC 2.3.3; on the platform, every accelerated layer output will be compared with the CPU result; runs will be deterministic. **Metrics.** End-to-end cycles per model, per-layer breakdown (prep, transfer, compute, CPU-only), multiplier utilization, bytes moved; for S5 also the CPU time free while waiting, bus wait cycles and the longest CPU stall. **Deliverables.** Open-source code and scripts, and a term paper with the literature review and the design insights.

References

★ = published in a venue on the instructor's list (Journal of Systems Architecture, IEEE Computer Architecture Letters, IEEE Transactions on Computers, ACM Transactions on Embedded Computing Systems, DATE).

[1] ★ F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, T. Krishna, "STONNE: Enabling cycle-level microarchitectural simulation for DNN inference accelerators," *IEEE Computer Architecture Letters*, vol. 20, no. 2, pp. 122–125, 2021. Code: github.com/stonne-simulator/stonne

[2] F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, T. Krishna, "STONNE: Enabling cycle-level microarchitectural simulation for DNN inference accelerators," *IEEE Int. Symp. on Workload Characterization (IISWC)*, 2021.

[3] ★ V. Herdt, D. Große, P. Pieper, R. Drechsler, "RISC-V based virtual prototype: An extensible and configurable platform for the system-level," *Journal of Systems Architecture*, vol. 109, art. 101756, 2020.

[4] ★ M. Schlägl, M. Stockinger, D. Große, "A RISC-V "V" VP: Unlocking vector processing for evaluation at the system level," *Design, Automation and Test in Europe (DATE)*, 2024.

[5] M. Schlägl, C. Hazott, D. Große, "RISC-V VP++: Next generation open-source virtual prototype," *Workshop on Open-Source Design Automation (OSDA)*, 2024. Code: github.com/ics-jku/riscv-vp-plusplus

[6] C. Banbury et al., "MLPerf Tiny benchmark," *NeurIPS Datasets and Benchmarks Track*, 2021. Code: github.com/mlcommons/tiny

[7] H. Kwon, A. Samajdar, T. Krishna, "MAERI: Enabling flexible dataflow mapping over DNN accelerators via reconfigurable interconnects," *ASPLOS*, 2018.

[8] E. Qin et al., "SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," *IEEE HPCA*, 2020.

[9] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," *ISCA*, 2017.

[10] J. Haris, P. Gibson, J. Cano, N. B. Agostini, D. Kaeli, "SECDA: Efficient hardware/software co-design of FPGA-based DNN accelerators for edge inference," *SBAC-PAD*, 2021.

[11] ★ H. Ham, W. Yang, Y. Shin, O. Woo, G. Heo, S. Lee, J. Park, G. Kim, "ONNXim: A fast, cycle-level multi-core NPU simulator," *IEEE Computer Architecture Letters*, 2024.

[12] ★ A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, F. Conti, "DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs," *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1253–1268, 2021.

[13] ★ L. Mei, P. Houshmand, V. Jain, S. Giraldo, M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1160–1174, 2021.

[14] ★ A. Garofalo, G. Tagliavini, F. Conti, D. Rossi, L. Benini, "XpulpNN: Accelerating quantized neural networks on RISC-V processors through ISA extensions," *Design, Automation and Test in Europe (DATE)*, 2020.

[15] ★ D. Giri, K.-L. Chiu, G. Di Guglielmo, P. Mantovani, L. P. Carloni, "ESP4ML: Platform-based design of systems-on-chip for embedded machine learning," *Design, Automation and Test in Europe (DATE)*, 2020.

[16] IEEE Std 1666-2011, "IEEE standard for standard SystemC language reference manual," 2012.